

Week 1: Data Structures And Algorithms

Exercise 2: Implementing the Factory Method Pattern

Sol.:

Big O Notation:

The Big O notation describes the upper bound of an algorithm's running time or space requirement in terms of input size n . It helps to evaluate and compare the efficiency of algorithms, especially for large data by providing the time or space required for running an algorithm with respect to the input size n .

For example:

- $O(1)$ – Constant time (fastest)
- $O(n)$ – Linear time (time grows with input size)
- $O(\log n)$ – Logarithmic time (fast for large n)
- $O(n^2)$ – Quadratic time (slow for large n)

Where n is the input size.

- **Best, Average, and Worst-Case Scenarios**
- **Best-case:** The best case describes the lowest time/space required by the algorithm.
For example: The desired element is found immediately in an array (e.g., first element).
- **Average-case:** The average case describes the expected time/space used for a typical input.
For example: The desired element is found somewhere in the middle of the array.
- **Worst-case:** The worst case describes the highest time/space required by the algorithm.
For example: The desired element is not found or is the last element in the array.

Code:-

```
import java.util.Arrays;
```

```
import java.util.Comparator;

class Product {

    int productId;

    String productName;

    String category;

    public Product(int productId, String productName, String category) {

        this.productId = productId;

        this.productName = productName;

        this.category = category;

    }

    public String toString() {

        return productId + " - " + productName + " (" + category + ")";

    }

}

public class Main {

    public static void main(String[] args) {

        Product[] products = {

            new Product(1, "Laptop", "Electronics"),

            new Product(2, "Smartphone", "Electronics"),

            new Product(3, "Tablet", "Electronics"),

            new Product(4, "Smartwatch", "Electronics"),

        }

    }

}
```

```

        new Product(5, "Bluetooth Speaker", "Electronics"),

        new Product(6, "Wireless Mouse", "Accessories"),

};

Product[] sortedProducts = products.clone();

Arrays.sort(sortedProducts, Comparator.comparing(p ->
p.productName.toLowerCase()));

Product result1 = linearSearch(products, "Wireless Mouse");

System.out.println("Linear Search Result: " + result1);

Product result2 = binarySearch(sortedProducts, "Wireless Mouse");

System.out.println("Binary Search Result: " + result2);

}

public static Product linearSearch(Product[] products, String name) {

    for (Product p : products) {

        if (p.productName.equalsIgnoreCase(name)) {

            return p;

        }

    }

    return null;

}

```

```

public static Product binarySearch(Product[] products, String name) {

    int left = 0, right = products.length - 1;

    name = name.toLowerCase();

    while (left <= right) {

        int mid = (left + right) / 2;

        String midName = products[mid].productName.toLowerCase();

        if (midName.equals(name)) return products[mid];

        if (midName.compareTo(name) < 0) left = mid + 1;

        else right = mid - 1;

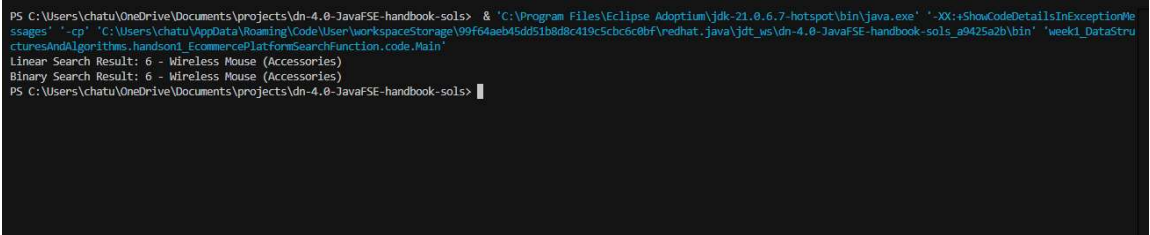
    }

    return null;

}
}

```

- Output



```

PS C:\Users\chatu\OneDrive\Documents\projects\dn-4.0-JavaFSE-handbook-sols> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.6.7-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chatu\AppData\Roaming\Code\User\workspaceStorage\99f64aeb45dd51b8d8c419c5cb6c0bf\redhat_java\jdt_ws\dn-4.0-JavaFSE-handbook-sols_a9425a2b\bin' 'week1_DataStructuresAndAlgorithms\handson1_EcommercePlatformSearchFunction.code.Main'
Linear Search Result: 6 - Wireless Mouse (Accessories)
Binary Search Result: 6 - Wireless Mouse (Accessories)
PS C:\Users\chatu\OneDrive\Documents\projects\dn-4.0-JavaFSE-handbook-sols>

```

- Time Complexity comparison
 1. Linear Search
 - Best Case - $O(1)$
 - Average Case - $O(n)$
 - Worst Case - $O(n)$

2. Binary Search

- Best Case - $O(1)$
- Average Case - $O(\log n)$
- Worst Case - $O(\log n)$